

Quiz Code de la Route — Présentation du projet

Application Android réalisée avec Android Studio : un quiz de 5 questions sur le code de la route. Ce projet combine base de données locale/serveur, authentification Firebase et affichage des scores — conçu pour démontrer des compétences en développement mobile et backend léger.



Objectifs pédagogiques

Concevoir une app Android

Architecture MVVM/Activity
→ Fragments, cycles de vie et UI réactive.

Gérer les données

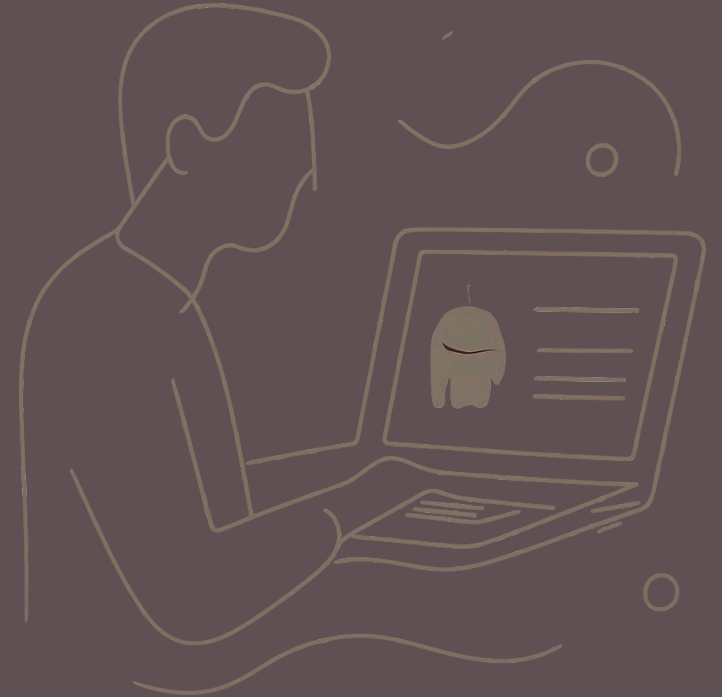
Stockage et récupération des questions via SQLite et serveur Node.js.

Authentification

Inscription / connexion avec Firebase (email + mot de passe).

Suivi des performances

Calcul et affichage d'un score sur 100% ; journalisation côté terminal.



Architecture technique — Vue d'ensemble

Flux principal : Application Android ↔ ngrok (tunnel) ↔ server.js (Node) ↔ SQLite. Firebase gère l'authentification utilisateur en parallèle. Les scores sont renvoyés à l'app et consignés localement dans le terminal du serveur pour chaque connexion.



Composants principaux



server.js (Node)

API REST simple : endpoints pour récupérer les questions et enregistrer les scores.



SQLite

Stockage des 5 questions, choix multiples et réponse correcte ; requêtes CRUD minimales.



ngrok

Tunnel sécurisé pour exposer le serveur local pendant le développement et les tests.

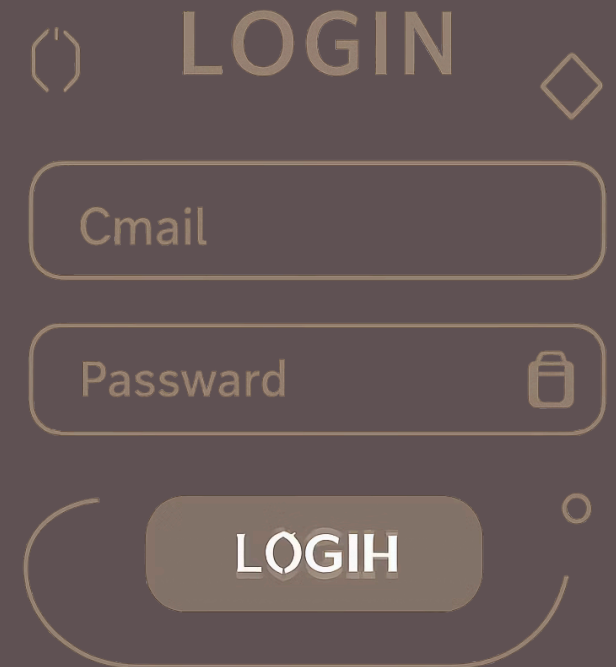


Firebase Auth

Inscription/connexion par e-mail et mot de passe ; sessions gérées côté client.

Authentification utilisateur (Firebase)

Fonctionnalités : création de compte (email + mot de passe), connexion sécurisée, gestion d'erreurs (mot de passe faible, email invalide). Après connexion, l'utilisateur accède au quiz et ses sessions sont liées à son compte pour tracer les scores.

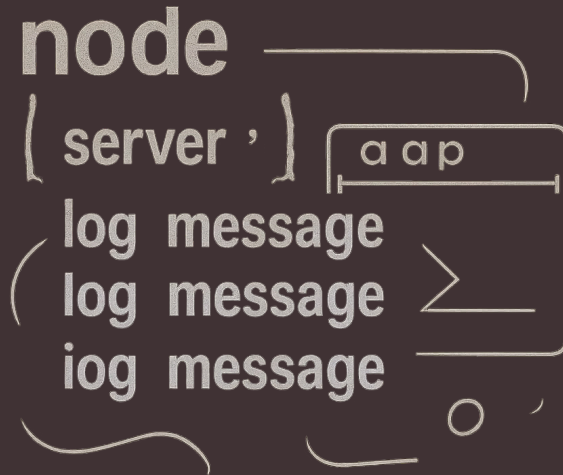


Design du quiz et UX

Structure : 5 questions à choix multiple, interface épurée, navigation contrôlée (suivant / précédent), validation immédiate de la réponse. À la fin : écran de score (valeur sur 100%) avec récapitulatif des réponses correctes et incorrectes.



Back-end et base de données



server.js expose :

- /questions — GET : retourne les 5 questions depuis SQLite
- /submit-score — POST : enregistre le score et retourne confirmation
- Journalisation : à chaque connexion, le terminal affiche l'utilisateur connecté et son score

① ngrok a permis d'exposer temporairement le serveur local pour tester l'app à partir d'un appareil physique.

Affichage et gestion des scores

Calcul : $\text{score} = \text{nombre de réponses correctes} \times 20$ (pour obtenir un résultat sur 100%). Le score est affiché dans l'app et envoyé au serveur. Le terminal (server.js) conserve un log par compte connecté : email + score, utile pour vérification et débogage.





Défis rencontrés & solutions

Synchronisation réseau

Problème :
latence via ngrok
→ Solution : mise
en cache local et
gestion d'erreurs
réseau.

Authentification

Problème :
gestion des
sessions et des
tokens →
Solution :
intégration du
SDK Firebase et
validation côté
client.

Intégrité des données

Problème :
collisions
d'écriture des
scores →
Solution :
vérification
serveur et
réponse
atomique à
l'insertion.

Conclusion & prochaines étapes

- Synthèse : application fonctionnelle démontrant intégration Android + Node.js + SQLite + Firebase.
- Améliorations proposées : ajouter un tableau de classement global, questions dynamiques depuis une interface d'administration, analytics simple pour suivre progrès.
- Livrables : code source, script d'initialisation SQLite, guide d'installation (ngrok + server), démonstration live.

✅ Prêt pour une démonstration en direct et pour des questions techniques des évaluateurs.

